

Group 57 : CS6310 Fall 2025 Assignment 1

Legend
RELATIONSHIPS
--> Directed association / uses
o.. Weak/loose association
o-- Aggregation
..> Dependency
*-- Composition
VISIBILITY & STEREOTYPES
+ public
- private
protected % not used due to lack of inheritance
<u>static</u>
<> enum stereotype
MULTIPLICITIES
"1" exactly one
"1..*" one or more
"0..*" zero or more

CommandManager
+ CommandManager() - outputMessageToStdOut(message: string) : : void - processCommand() : : void

BattleManager
- seed: int - pokemonActive: List<Pokemon> - battleBuffer: string
+ BattleManager() + startBattle(poke1Name: string, poke2Name: string) : : string %% returns the battleBuffer + stopBattle() : : void + setSeed(seed: int) : : void + removeSeed() : : void - checkExistsPokemon(pokeName: string) : : bool - addOutputToBuffer(currMoveOutput: string) : : void - clearBuffer() : : void - iterativeBattle(poke1Name: string, poke2Name: string) : : int

PokeDex
<i>Singleton responsible for abstracting out all smart Pokemon creation logic. Either reaches out to predetermined list or some dynamic API for gathering Pokemon attributes.</i> - instance: PokeDex - pokemonList: List<Pokemon> % This is allocated on construct with all pokemon and all their available moves - moveList: List<Moves> % This is a list of all possible moves, used by Custom Pokemon
+ getInstance() : : PokeDex + getPokemonByName(pokeName: string) : : Pokemon % Checks list of pokemon, if doesn't exist then create Custom Pokemon - PokeDex()

Pokemon
- name: string - seed: int - stat: Stat - winCount: int - lossCount: int - moves: List<Moves> - moveBuffer: Move % defensive moves is stored in the moveBuffer to check during next applyMove
+ incrementWins() : : void + incrementLosses() : : void + applyMove(attackingMove: Move) : : string + setSeed(seed: int) : : void + checkAlive() : : bool + getMove() : : Move + rest() : : void + getName() : : string - calculateMoveType() : : MoveType - getRandomMove(moveType: MoveType) : : Move - setMoveBuffer(previousMove: Move) : : void

Stat
- currentHP: int - maxHP: int
+ getCurrentHP() : : int + decrementCurrentHP(subtract: int) : : void + resetCurrentToMax() : : void

Move
- power: int - moveType: MoveType
+ getPower() : : int + getMoveType() : : MoveType

«enumeration» MoveType
Attack Defensive

